



2장

실습 환경 설정과 파이토치 기초

2장 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

2.2 파이토치 기초 문법

2.3 실습 환경 설정(생략)

2.4 파이토치 코드 맛보기

2.1 파이토치 개요



2.1 파이토치 개요

● 파이토치 개요

- 파이토치(PyTorch): 2017년 초 공개된 딥러닝 프레임워크로, 루아 기반 Torch를 파이썬으로 발전시킨 것
- Torch의 배경: 초기에는 NumPy처럼 과학 연산 라이브러리 성격이었으나, 이후 딥러닝 프레임워크로 발전
- (공식 튜토리얼 관점) 파이토치의 대상/용도
 - NumPy를 대체하면서 GPU 연산이 필요한 경우
 - 유연성과 속도가 높은 딥러닝 연구 플랫폼이 필요한 경우
- 장점: 간결하고 빠르게 구현할 수 있어 많이 사용됨



2.1 파이토치 개요

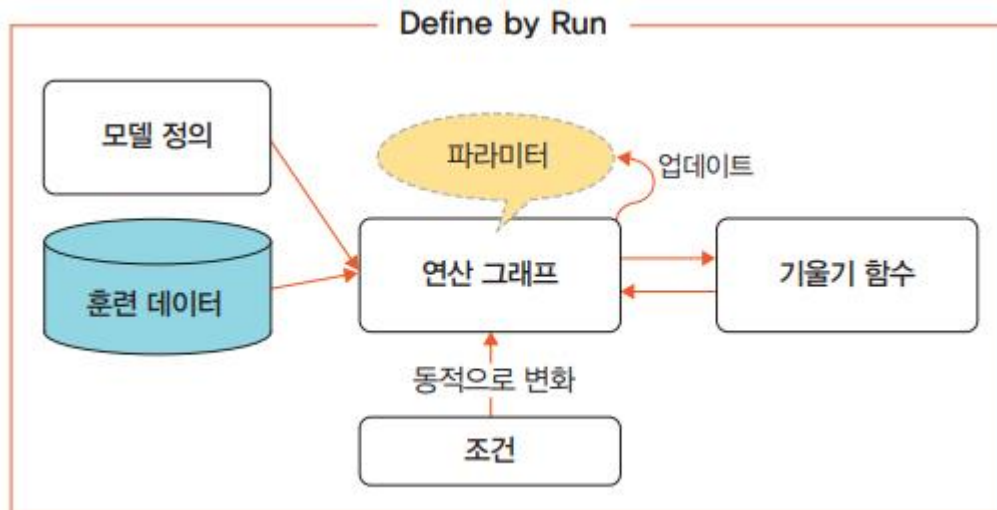
● 파이토치 특징 및 장점

- 질문: GPU, 텐서, 동적 신경망의 의미를 이해하자
 - GPU(Graphics Processing Unit)
 - 대규모 병렬 연산으로 연산 속도를 크게 향상
 - 딥러닝에서 기울기(미분) 계산을 빠르게 처리하는 데 유리
 - CUDA, cuDNN 같은 API/라이브러리를 통해 GPU 연산을 사용
 - 병렬 처리 성능이 CPU보다 훨씬 좋아 딥러닝 학습에서 필수적이라고 볼 수 있음
- 텐서(Tensor)
 - 파이토치의 기본 데이터 형태
 - 같은 자료형으로 이루어진 다차원 배열(행렬)
 - `.cuda()` 같은 간단한 명령으로 GPU 연산으로 전환 가능
- 동적 신경망(Dynamic Neural Network)
 - 학습 반복마다 네트워크 구조 변경이 가능한 신경망
 - 예: 학습 중 은닉층 추가/제거 등 구조 조작 가능
 - Define by Run 방식: 실행하면서 연산 그래프가 정의되어 그래프/연산을 분리해 생각할 필요가 없어 코드 이해가 쉬움



2.1 파이토치 개요

▼ 그림 2-1 파이토치 'Define by Run'





2.1 파이토치 개요

● 파이토치 특징 및 장점

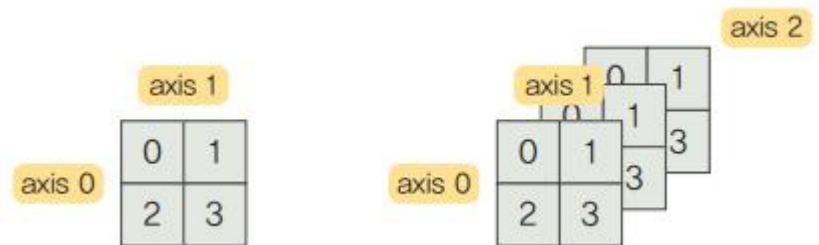
벡터, 행렬, 텐서

- 머신러닝/딥러닝에서 데이터는 보통 벡터 형태로 표현함
- 벡터(Vector): [1.0, 1.1, 1.2]처럼 숫자 리스트로 된 1차원 배열
- 행렬(Matrix): 행(row)과 열(column)로 구성된 2차원 배열
 - 가로줄 = 행(row), 세로줄 = 열(column)
- 텐서(Tensor): 3차원 이상의 배열(다차원 배열)

● 축(axis) 정리

- axis 0: 1차원 축 → 벡터
- axis 1: 2차원 축 → 행렬
- axis 2: 3차원 축(채널) → 텐서

▼ 그림 2-2 벡터, 행렬, 텐서





2.1 파이토치 개요

● 파이토치 특징 및 장점

- 행렬의 일반적인 표현은 다음과 같음

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

- 텐서는 행렬의 다차원 표현이라고 생각하면 쉬움
- 같은 크기의 행렬이 여러 개 묶여 있는 것으로 다음과 같이 표현할 수 있음

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$$



2.1 파이토치 개요

● 파이토치 특징 및 장점

- 파이토치에서 텐서를 표현하기 위해서는 다음 코드와 같이 `torch.tensor()`를 사용

```
import torch
torch.tensor([[1., -1.], [1., -1.]])
```

- 생성된 텐서의 형태는 다음과 같이 표현

```
tensor([[ 1., -1.],
        [ 1., -1.]])
```

- 벡터, 행렬 등 자세한 내용은 선형대수학 도서를 참고



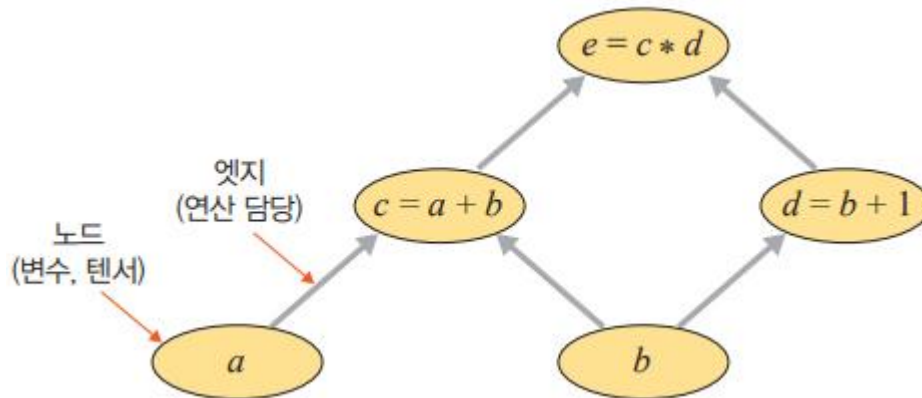
2.1 파이토치 개요

● 파이토치 특징 및 장점

연산 그래프

- 연산 그래프는 방향성이 있으며 변수(예 텐서)를 의미하는 노드와 연산(예 곱하기, 더하기)을 담당하는 엣지로 구성
- 다음 그림과 같이 노드는 변수(a , b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c , d , e)를 구성할 수 있음

▼ 그림 2-3 파이토치 연산 그래프





2.1 파이토치 개요

● 파이토치 특징 및 장점

- 신경망 학습은 연산 그래프를 기반으로 계산됨
 - 손실 함수의 기울기(gradient)를 가중치·바이어스에 대해 계산
 - 경사 하강법으로 가중치가 업데이트됨
 - 연산 그래프를 통해 이 과정이 효율적으로 수행됨

● 파이토치의 특징/장점

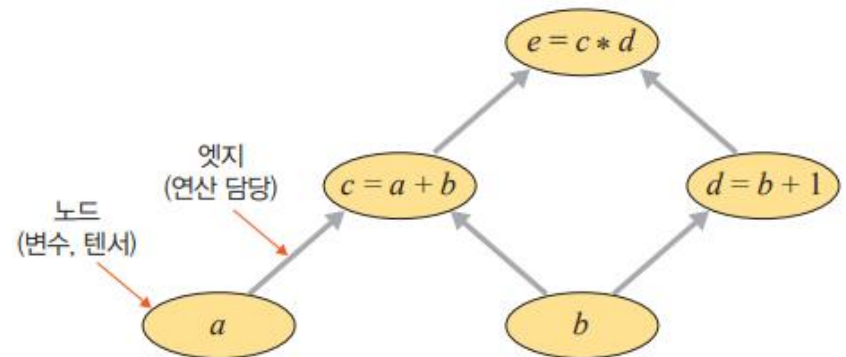
- 단순함(효율적 계산)
- 파이썬 환경과 통합이 쉬움
- 디버깅이 직관적이고 코드가 간결함

● 성능(낮은 CPU 활용)

- 모델 훈련 시 CPU 사용률이 비교적 낮음
- 학습/추론 속도가 빠르고 사용이 쉬움

● 직관적 인터페이스

- 텐서플로처럼 API가 자주 크게 바뀌지 않아 학습/사용이 쉬움

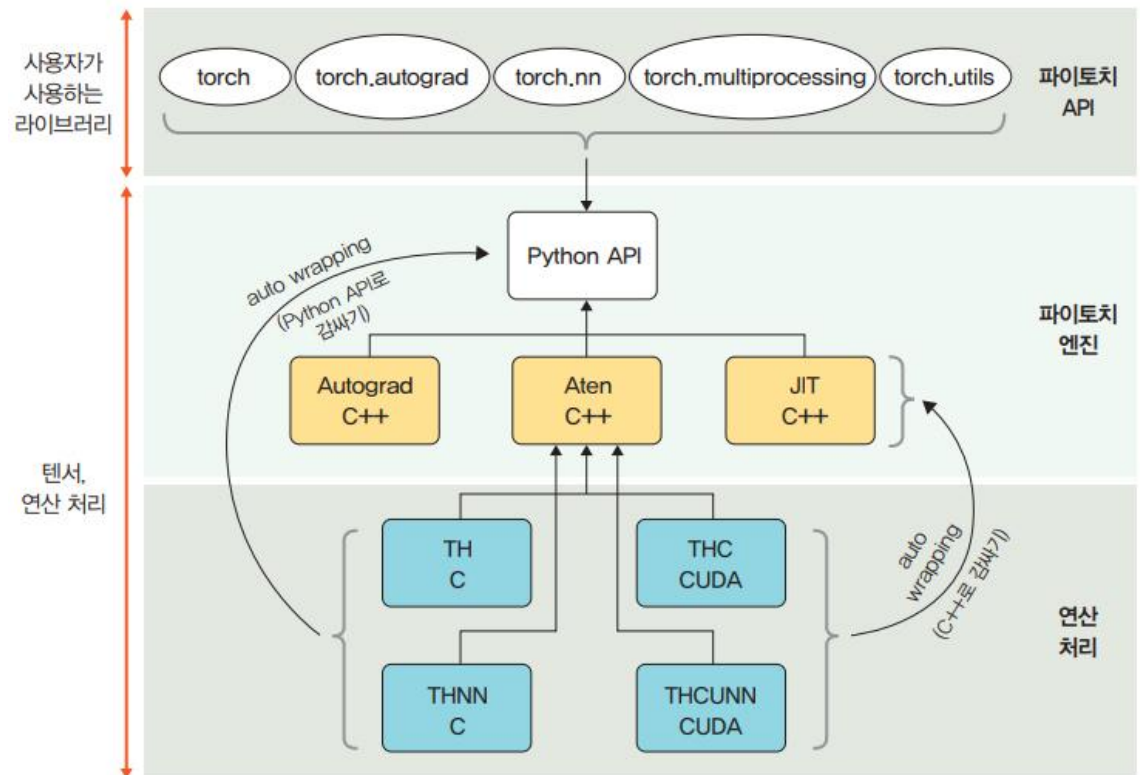




2.1 파이토치 개요

● 파이토치의 아키텍처

- 파이토치 아키텍처 구성: PyTorch API / PyTorch 엔진 / 연산 처리(백엔드)
- PyTorch 엔진: 다차원 텐서 연산과 자동 미분(autograd)을 담당
- 연산 처리(하단 백엔드): 텐서의 실제 연산을 수행하는 계층
- C, CUDA 라이브러리: CPU/GPU에서의 실질적인 계산을 담당하는 핵심 라이브러리 위치



▼ 그림 2-4 파이토치의 아키텍처

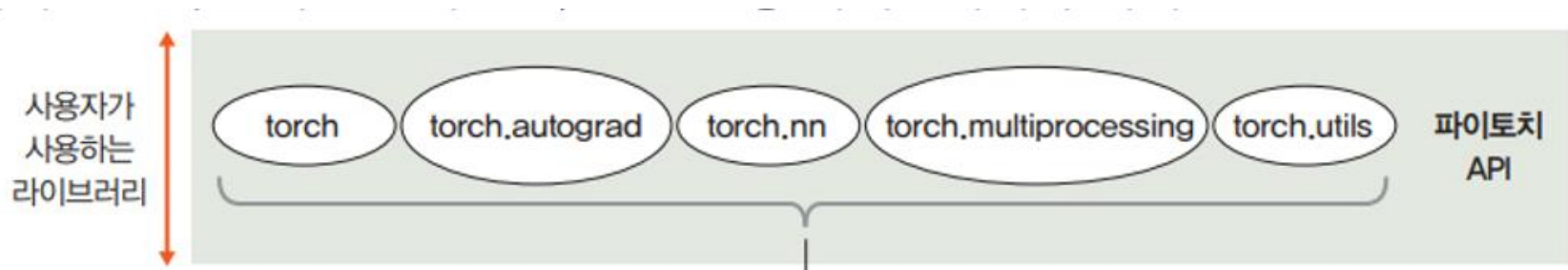


2.1 파이토치 개요

● 파이토치의 아키텍처

파이토치 API

- 파이토치 API 계층 역할
 - 사용자가 이해하기 쉬운 API를 제공해 텐서 처리, 신경망 구축/훈련을 돕는다.
- 계산 수행 여부
 - 이 계층은 사용자 인터페이스를 제공하지만 실제 연산은 수행하지 않는다.
- 작업 전달 구조
 - 요청된 작업을 C++로 구현된 파이토치 엔진에 전달해 실행하게 한다.
- 추가 제공 요소
 - 사용자 편의성을 위해 여러 패키지(모듈)가 함께 제공된다.



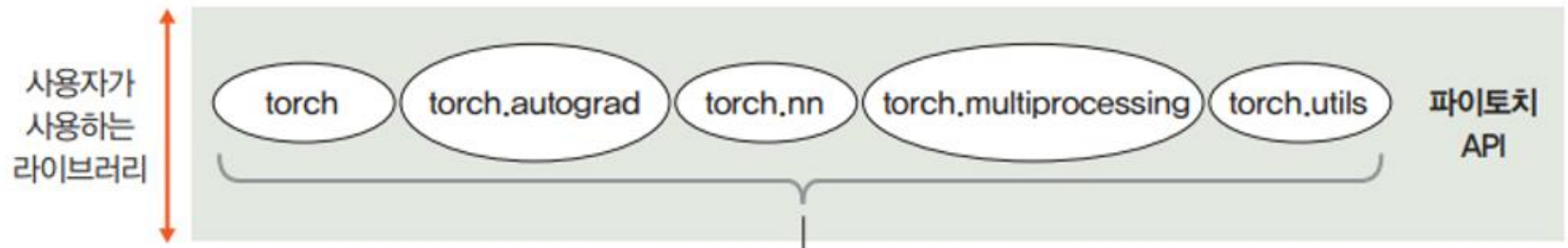


2.1 파이토치 개요

● 파이토치의 아키텍처

torch: GPU를 지원하는 텐서 패키지

- 다차원 텐서를 기반으로 다양한 수학적 연산이 가능하도록 함
- CPU뿐만 아니라 GPU에서 연산이 가능하므로 빠른 속도로 많은 양의 계산을 할 수 있음



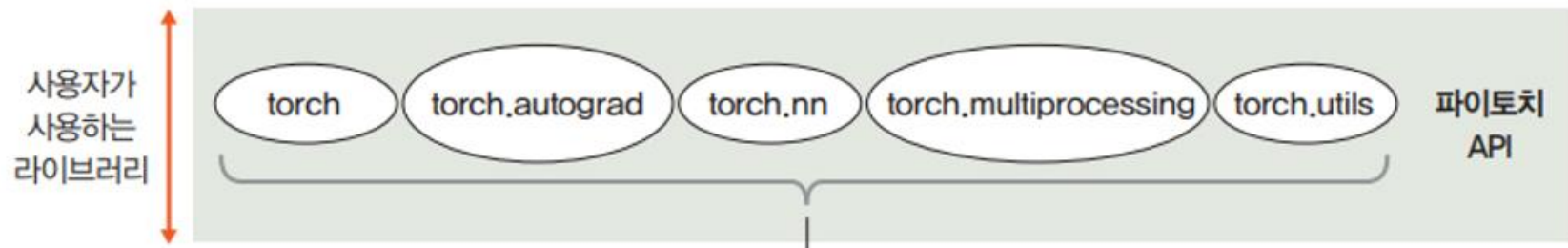


2.1 파이토치 개요

● 파이토치의 아키텍처

torch.autograd: 자동 미분 패키지

- 다른 프레임워크와의 차별점: 파이토치를 대표하는 핵심 기능 중 하나로 유연한 미분 계산을 지원
- 일반적인 한계: 신경망 구조를 조금만 바꿔도(예: 은닉층 노드 수 변경) 다시 구성해야 하는 경우가 많음
- 파이토치 방식: 자동 미분으로 미분(기울기) 계산을 효율적으로 처리
- 결과: 연산 그래프가 즉시(실행 시점에) 생성/계산되어, 네트워크 수정이 바로 반영되고 다양한 모델을 쉽게 실험할 수 있음





2.1 파이토치 개요

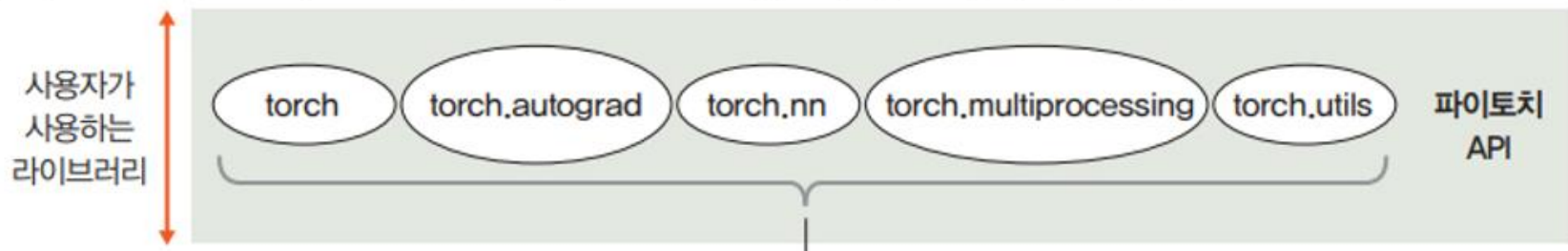
● 파이토치의 아키텍처

torch.nn: 신경망 구축 및 훈련 패키지

- torch.nn을 사용할 경우 신경망을 쉽게 구축하고 사용할 수 있음
- 합성곱 신경망, 순환 신경망, 정규화 등이 포함되어 손쉽게 신경망을 구축하고 학습시킬 수 있음

torch.multiprocessing: 파이썬 멀티프로세싱 패키지

- 파이토치에서 사용하는 프로세스 전반에 걸쳐 텐서의 메모리 공유가 가능
- 서로 다른 프로세스에서 동일한 데이터(텐서)에 대한 접근 및 사용이 가능



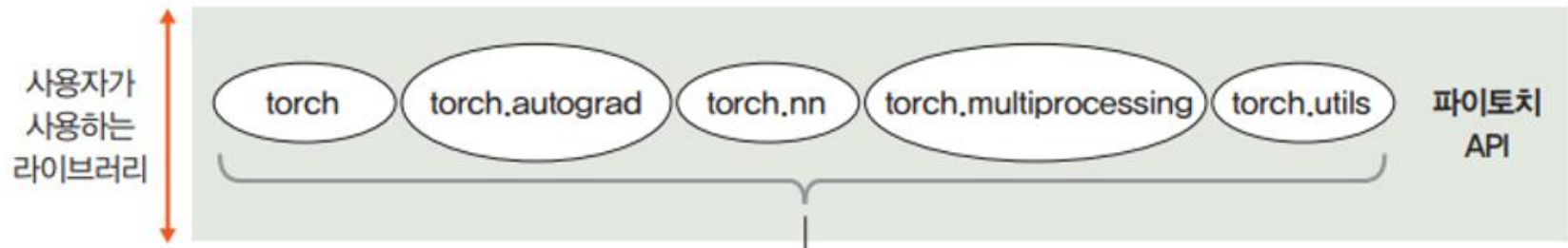


2.1 파이토치 개요

● 파이토치의 아키텍처

torch.utils: DataLoader 및 기타 유틸리티를 제공하는 패키지

- 모델에 데이터를 제공하기 위한 torch.utils.data.DataLoader 모듈을 주로 사용
- 병목 현상을 디버깅하기 위한 torch.utils.bottleneck, 모델 또는 모델의 일부를 검사하기 위한 torch.utils.checkpoint 등의 모듈도 있음



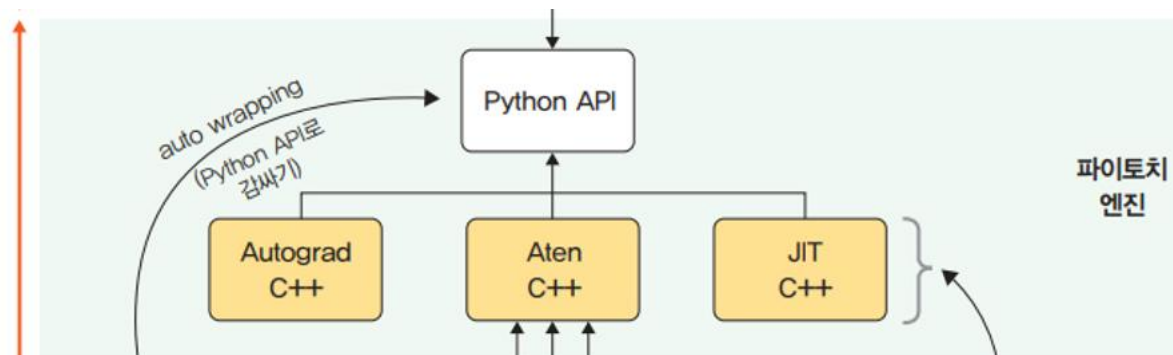


2.1 파이토치 개요

● 파이토치의 아키텍처

파이토치 엔진

- 파이토치 엔진 구성: Autograd C++, ATen C++, JIT C++, Python API
- Autograd C++: 학습 시 가중치/바이어스 업데이트에 필요한 미분(기울기)을 자동 계산
- ATen C++: 파이토치의 핵심 C++ 텐서 연산 라이브러리 제공
- JIT C++: 연산을 더 빠르게 하기 위한 JIT(Just-In-Time) 컴파일/최적화 담당
- 래핑 구조: 내부는 C++로 구현되지만 Python API로 감싸 제공되어 사용자는 쉽게 모델/텐서를 사용 가능



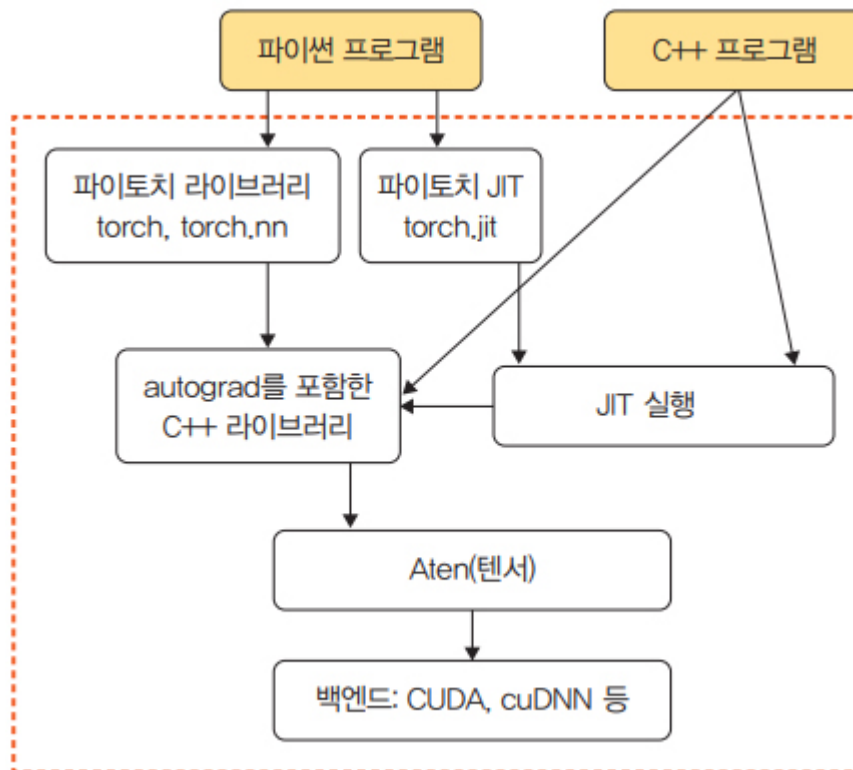


2.1 파이토치 개요

● 파이토치의 아키텍처

- 이들의 관계를 도식화하면 다음 그림과 같음

▼ 그림 2-5 파이토치 엔진



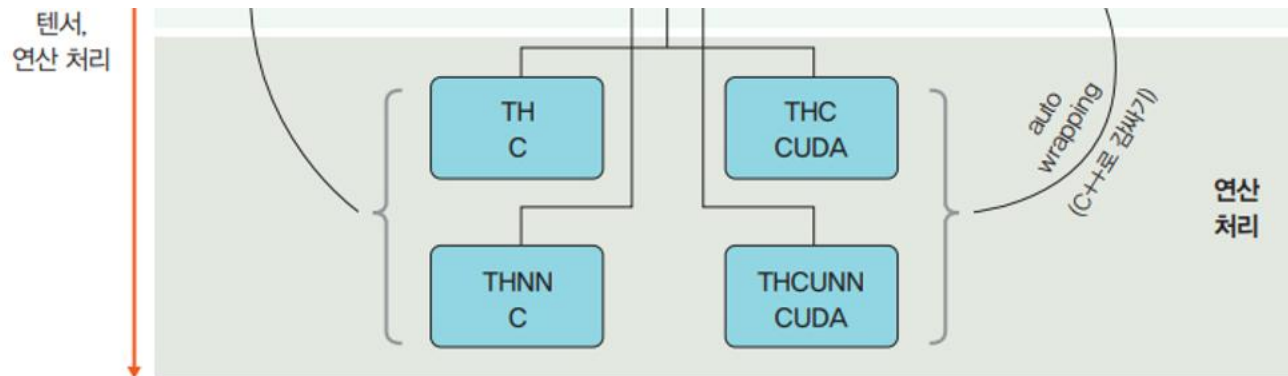


2.1 파이토치 개요

● 파이토치의 아키텍처

연산 처리

- 가장 아래 계층에 속하는 C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산을 수행
- 여기에서 제공되는 패키지는 CPU와 GPU(TH(토치), THC(토치 CUDA))를 이용하여 효율적인 데이터 구조, 다차원 텐서에 대한 연산을 처리



2.2 파이토치 기초 문법

교재 사이트에서 예제 다운로드 받기

딥러닝
파이토치
교과서



딥러닝 파이토치 교과서 사이트

The screenshot shows the GitHub interface for the repository 'gilbutITbook / 080289'. The left sidebar displays the file structure with folders 'chap02' through 'chap13' and files 'colab_2장.ipynb' and 'python_2장.ipynb'. The main content area shows the '080289 / chap02' directory with a table of files and their commit messages.

Name	Last commit message
..	
data	initial release
colab_2장.ipynb	initial release
python_2장.ipynb	initial release



2.2 파이토치 기초 문법

● 텐서 다루기

텐서 생성 및 변환

- 텐서는 파이토치의 가장 기본이 되는 데이터 구조
- 넘파이의 ndarray와 비슷하며 GPU에서의 연산도 가능
- 텐서 생성은 다음과 같은 코드를 이용

```
import torch
print(torch.tensor([[1,2],[3,4]])) ----- 2차원 형태의 텐서 생성
print(torch.tensor([[1,2],[3,4]], device="cuda:0")) ----- GPU에 텐서 생성
print(torch.tensor([[1,2],[3,4]], dtype=torch.float64)) ----- dtype을 이용하여 텐서 생성
```



2.2 파이토치 기초 문법

- 텐서 다루기

- 다음은 생성된 텐서의 결과

```
tensor([[1, 2],
        [3, 4]])
```

```
tensor([[1., 2.],
        [3., 4.]], dtype=torch.float64)
```



2.2 파이토치 기초 문법

● 텐서 다루기

- 이번에는 텐서를 ndarray로 변환해 보자

```
temp = torch.tensor([[1,2],[3,4]])
print(temp.numpy()) ----- 텐서를 ndarray로 변환
```

```
temp = torch.tensor([[1,2],[3,4]], device="cuda:0")
print(temp.to("cpu").numpy()) ----- GPU상의 텐서를 CPU의 텐서로 변환한 후 ndarray로 변환
```



2.2 파이토치 기초 문법

- 텐서 다루기

- 다음은 텐서를 ndarray로 변환한 결과

```
[[1 2]
 [3 4]]
```

```
[[1 2]
 [3 4]]
```



2.2 파이토치 기초 문법

● 텐서 다루기

텐서의 인덱스 조작

- 텐서 인덱스 조작 방법은 여러 가지가 있음
- PyTorch 텐서는 NumPy ndarray처럼 동작하여
 - 인덱스를 직접 지정하거나
 - 슬라이스(slicing) 등을 사용해 데이터 선택/조작 가능

● 주요 텐서 자료형 예시

- torch.FloatTensor: 32-bit 부동소수점
- torch.DoubleTensor: 64-bit 부동소수점
- torch.LongTensor: 64-bit 부호 있는 정수



2.2 파이토치 기초 문법

● 텐서 다루기

- 이외에도 다양한 유형의 텐서가 있음
- 텐서의 인덱스 조작은 다음과 같은 코드를 이용

```
temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7]) ----- 파이토치로 1차원 벡터 생성
print(temp[0], temp[1], temp[-1]) ----- 인덱스로 접근
print('-----')
print(temp[2:5], temp[4:-1]) ----- 슬라이스로 접근
```

- 코드에서 -1번 인덱스는 맨 뒤에서부터 시작하는 인덱스를 의미
- 다음은 인덱스 조작에 대한 결과

```
tensor(1.) tensor(2.) tensor(7.)
-----
tensor([3., 4., 5.]) tensor([5., 6.])
```



퀴즈1

- 출력 결과는?

```
import torch
```

```
temp = torch.FloatTensor([10, 20, 30, 40, 50, 60, 70, 80])
```

```
print(temp[2], temp[-3])
```

```
print('-----')
```

```
print(temp[1:6], temp[::2])
```

```
print('-----')
```

```
print(temp[-5:-1], temp[3:])
```



2.2 파이토치 기초 문법

● 텐서 다루기

텐서 연산 및 차원 조작

- 텐서는 넘파이의 ndarray처럼 다양한 수학 연산이 가능하며, GPU를 사용하면 더 빠르게 연산할 수 있음
- 참고로 텐서 간의 타입이 다르면 연산이 불가능함
- 예를 들어 FloatTensor와 DoubleTensor 간에 사칙 연산을 수행하면 오류가 발생
- 다음과 같이 벡터 두 개를 생성하여 사칙 연산을 할 수 있음

```
v = torch.tensor([1, 2, 3]) ----- 길이가 3인 벡터 생성
w = torch.tensor([3, 4, 6])
print(w - v) ----- 길이가 같은 벡터 간 뺄셈 연산
```



2.2 파이토치 기초 문법

- 텐서 다루기
 - 다음은 벡터 간 뿔셈 연산에 대한 결과
`tensor([2, 2, 3])`



2.2 파이토치 기초 문법

● 텐서 다루기

- 텐서의 차원을 조작하는 코드는 다음과 같음

```
temp = torch.tensor([
    [1, 2], [3, 4]]) ----- 2x2 행렬 생성
```

```
print(temp.shape)
```

```
print('-----')
```

```
print(temp.view(4, 1)) ----- 2x2 행렬을 4x1로 변형
```

```
print('-----')
```

```
print(temp.view(-1)) ----- 2x2 행렬을 1차원 벡터로 변형
```

```
print('-----')
```

```
print(temp.view(1, -1)) ----- -1은 (1, ?)와 같은 의미로 다른 차원으로부터 해당 값을 유추하겠다는
```

```
print('-----')
```

```
print(temp.view(-1, 1)) ----- 앞에서와 마찬가지로 (?, 1)의 의미로 temp의 원소 개수(2x2=4)를  
유지한 채 (?, 1)의 형태를 만족해야 하므로 (4, 1)이 됩니다.
```

것입니다. temp의 원소 개수(2x2=4)를 유지한 채 (1, ?)의 형태를 만족해야 하므로 (1, 4)가 됩니다.



2.2 파이토치 기초 문법

● 텐서 다루기

- 다음은 텐서의 차원을 조작한 결과

```
torch.Size([2, 2])
```

```
-----
tensor([[1],
        [2],
        [3],
        [4]])
```

```
-----
tensor([1, 2, 3, 4])
```

```
-----
tensor([[1, 2, 3, 4]])
```

```
-----
tensor([[1],
        [2],
        [3],
        [4]])
```



2.2 파이토치 기초 문법

- 데이터 준비
 - 데이터 호출 방법
 - Pandas 같은 파이썬 라이브러리로 불러오기
 - PyTorch 제공 데이터/도구를 이용해 불러오기
 - 데이터가 이미지인 경우
 - 파일에서 데이터를 읽기 → 전처리 → 배치 단위로 분할하여 처리
 - 데이터가 텍스트인 경우
 - 임베딩 수행 → 길이가 다른 시퀀스를 배치 단위로 분할하여 처리



2.2 파이토치 기초 문법

● 모델 정의

- 파이토치에서 모델을 정의하기 위해서는 모듈(module)을 상속한 클래스를 사용
- 그렇다면 모델과 모듈은 무엇이 다를까?
 - 계층(layer): 모듈 또는 모듈을 구성하는 한 개의 계층으로 합성곱층(convolutional layer), 선형 계층(linear layer) 등이 있음
 - 모듈(module): 한 개 이상의 계층이 모여서 구성된 것으로, 모듈이 모여 새로운 모듈을 만들 수도 있음
 - 모델(model): 최종적으로 원하는 네트워크로, 한 개의 모듈이 모델이 될 수도 있음



2.2 파이토치 기초 문법

● 모델 정의

`nn.Module()`을 상속하여 정의하는 방법

- 파이토치에서 `nn.Module`을 상속받는 모델은 기본적으로 `__init__()`과 `forward()` 함수를 포함
- `__init__()`에서는 모델에서 사용될 모듈(`nn.Linear`, `nn.Conv2d`), 활성화 함수 등을 정의하고, `forward()` 함수에서는 모델에서 실행되어야 하는 연산을 정의



2.2 파이토치 기초 문법

● 모델 정의

- 다음은 파이토치에서 모델을 정의하는 코드

```
class MLP(Module):
    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer = Linear(inputs, 1) ----- 계층 정의
        self.activation = Sigmoid() ----- 활성화 함수 정의

    def forward(self, X):
        X = self.layer(X)
        X = self.activation(X)
        return X
```



2.2 파이토치 기초 문법

● 모델 정의

Sequential 신경망을 정의하는 방법

- nn.Sequential을 사용하면 `__init__()`에서 **레이어(모듈)**을 **순서대로 정의**할 수 있음
- `forward()`에서 각 연산을 일일이 작성하지 않아도 되어 **코드 가독성이 좋아짐**
- Sequential 객체는 내부에 넣은 모듈들을 **등록된 순서대로 자동 실행**함

```
import torch.nn as nn
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=64, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))
```



2.2 파이토치 기초 문법

● 모델 정의

```
self.layer2 = nn.Sequential(
    nn.Conv2d(in_channels=64, out_channels=30, kernel_size=5),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2))
```

```
self.layer3 = nn.Sequential(
    nn.Linear(in_features=30*5*5, out_features=10, bias=True),
    nn.ReLU(inplace=True))
```

```
def forward(self, x):
    x = self.layer1(x)
    x = self.layer2(x)
    x = x.view(x.shape[0], -1)
```



2.2 파이토치 기초 문법

● 모델 정의

```

        x = self.layer3(x)
        return x

model = MLP() ----- 모델에 대한 객체 생성

print("Printing children\n-----")
print(list(model.children()))
print("\n\nPrinting Modules\n-----")
print(list(model.modules()))

```



2.2 파이토치 기초 문법

● 모델 정의

- 이 코드를 실행하면 다음과 같이 출력

Printing children

```
[Sequential(
  (0): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))
  (1): ReLU(inplace=True)
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
), Sequential(
  (0): Conv2d(64, 30, kernel_size=(5, 5), stride=(1, 1))
  (1): ReLU(inplace=True)
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
), Sequential(
  (0): Linear(in_features=750, out_features=10, bias=True)
  (1): ReLU(inplace=True)
)]
```



2.2 파이토치 기초 문법

● 모델 정의

Printing Modules

```
-----
[MLP(
  (layer1): Sequential(
    (0): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer2): Sequential(
    (0): Conv2d(64, 30, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer3): Sequential(
    (0): Linear(in_features=750, out_features=10, bias=True)
    (1): ReLU(inplace=True)
```



2.2 파이토치 기초 문법

● 모델 정의

```

    )
), Sequential(
    (0): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
), Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1)), ReLU(inplace=True),
MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False),
Sequential(
    (0): Conv2d(64, 30, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
), Conv2d(64, 30, kernel_size=(5, 5), stride=(1, 1)), ReLU(inplace=True),
MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False),
Sequential(
    (0): Linear(in_features=750, out_features=10, bias=True)
    (1): ReLU(inplace=True)
), Linear(in_features=750, out_features=10, bias=True), ReLU(inplace=True)]

```



2.2 파이토치 기초 문법

- 모델 정의

- `nn.Sequential`은 모델의 계층이 복잡할수록 효과가 뛰어남



2.2 파이토치 기초 문법

● 모델 정의

- 복잡한 모델(분기/재사용/스킵 연결 등)은 nn.Module을 상속받아 구현하는 것이 관리와 확장에 유리함

```
def MLP(in_features=1, hidden_features=20, out_features=1):
    hidden = nn.Linear(in_features=in_features, out_features=hidden_features,
                        bias=True)

    activation = nn.ReLU()
    output = nn.Linear(in_features=hidden_features, out_features=out_features,
                       bias=True)

    net = nn.Sequential(hidden, activation, output)
    return net
```

- ReLU, Softmax, Sigmoid 같은 활성화 함수는 모델을 정의할 때 레이어 사이에 포함하여 사용함



2.2 파이토치 기초 문법

● 모델의 파라미터 정의

- 모델 학습 전에 필요한 파라미터를 **미리 정의**해야 함
- **손실 함수(Loss function)**
 - 학습 중 **모델 출력값과 실제값(정답)** 사이의 오차를 측정
 - 예: $w x + b$ 의 예측값과 실제 y 의 차이를 통해 정확도/성능을 판단

● 자주 쓰는 손실 함수 예시

- BCELoss(Binary Cross-Entropy Loss): 이진 분류
- CrossEntropyLoss: 다중 클래스 분류
- MSELoss(Mean Squared Error Loss): 회귀 모델



2.2 파이토치 기초 문법

● 모델의 파라미터 정의

- 옵티마이저(optimizer): 데이터와 손실 함수를 바탕으로 **모델 파라미터(가중치/바이어스) 업데이트 방법**을 결정

다음은 옵티마이저의 주요 특성

- `step()`: 전달받은 파라미터를 업데이트
- 파라미터 그룹별로 서로 다른 학습률 등 설정 적용 가능
- `torch.optim.Optimizer(params, defaults)`: 모든 옵티마이저의 기본(부모) 클래스
- `zero_grad()`: 이전 스텝의 기울기(`gradient`)를 0으로 초기화
- `torch.optim.lr_scheduler`: 에포크 등에 따라 학습률을 조절할 수 있음



2.2 파이토치 기초 문법

- 모델의 파라미터 정의

- 지표(metrics): 훈련과 테스트 단계를 모니터링함
 - 2주차 평가는 accuracy(필수) + classification_report(선택)로 확인
 - 지표 상세(precision/recall/F1)는 후반 주차에서 다룸



지난 시간 이어서...



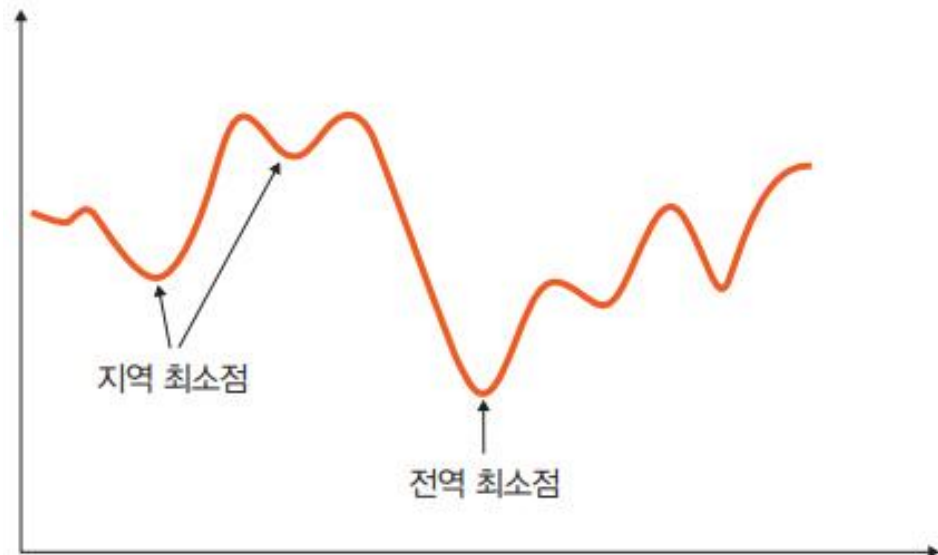
2.2 파이토치 기초 문법

● 모델의 파라미터 정의

전역 최소점과 최적점

- 손실 함수: 실제값과 예측값의 차이(오차)를 수치화
- 오차 $\uparrow \rightarrow$ 손실 값 $\uparrow$ / 오차 $\downarrow \rightarrow$ 손실 값 $\downarrow$
- 학습 목표: 손실 값을 최소화하는 가중치·바이어스 찾기
- 전역 최소점(Global minimum): 오차가 가장 작은 최적점(최종 목표)
- 지역 최소점(Local minimum): 최적점 탐색 중 만나는 함정으로, 빠지면 최적해를 못 찾을 수 있음

▼ 그림 2-11 전역 최소점과 최적점





2.2 파이토치 기초 문법

▼ 그림 2-12 파이토치 학습 절차

딥러닝 학습 절차	파이토치 학습 절차
모델, 손실 함수, 옵티마이저 정의	모델, 손실 함수, 옵티마이저 정의
전방향 학습(입력 → 출력 계산)	<code>optimizer.zero_grad():</code> 전방향 학습, 기울기 초기화
손실 함수로 출력과 정답의 차이(오차) 계산	<code>output = model(input):</code> 출력 계산
역전파 학습(기울기 계산)	<code>loss = loss_fn(output, target):</code> 오차 계산
기울기 업데이트	<code>loss.backward():</code> 역전파 학습
	<code>optimizer.step():</code> 기울기 업데이트

↑
모델 학습 과정



2.2 파이토치 기초 문법

● 모델 훈련

- `loss.backward()`를 사용해 기울기(`gradient`)를 자동으로 계산함
- 배치 학습이 반복되면 기울기(미분 값)가 누적될 수 있으므로, 매 반복마다 `zero_grad()`로 기울기를 0으로 초기화해야 함
- 이후 모델 훈련 예시 코드를 통해 전체 학습 흐름을 보여줌

```
for epoch in range(100):
    yhat = model(x_train)
    loss = criterion(yhat, y_train)
    optimizer.zero_grad() ----- 오차가 중첩적으로 쌓이지 않도록 초기화
    loss.backward()
    optimizer.step()
```



2.2 파이토치 기초 문법

- 모델 평가(Validation/Test)
 - `model.eval()` : 평가 모드 전환(드롭아웃/BN 영향 제거)
 - `with torch.no_grad()` : 역전파/기울기 저장 X (빠르고 메모리 절약)
 - `pred = output.argmax(dim=1)` : 예측 클래스 선택
 - `accuracy(필수) + classification_report(선택)`로 결과 확인



2.2 파이토치 기초 문법

- 훈련/평가 모드 전환으로 학습 과정을 관리: `model.train()` / `model.eval()`
- `model.train()`
 - 훈련 데이터에 사용, 모델을 학습 모드로 설정
 - Dropout 활성화(및 BatchNorm 등도 학습 모드로 동작)
- `model.eval()`
 - 검증/테스트 데이터에 사용, 모델을 평가 모드로 설정
 - Dropout 비활성화(모든 노드/유닛을 사용해 안정적으로 평가)



2.2 파이토치 기초 문법

● 훈련 과정 모니터링

- `model.train()`과 `model.eval()`을 상황에 맞게 설정하면 훈련/평가 동작이 올바르게 적용되어 성능(정확도) 관리에 도움이 됨
- `model.train()`은 앞에서 사용해 보았으니, 이번에는 `model.eval()`에 대한 사용 방법을 알아보자

```
model.eval() ----- 검증 모드로 전환(dropout=False)
with torch.no_grad(): ----- ①
    valid_loss = 0

    for x, y in valid_dataloader:
        outputs = model(x)
        loss = F.cross_entropy(outputs, y.long().squeeze())
        valid_loss += float(loss)
        y_hat += [outputs]

valid_loss = valid_loss / len(valid_loader)
```

- ① `model.eval()`에서 `with torch.no_grad()`를 사용하는 이유
 - 검증/테스트에서는 역전파가 필요 없으므로 기울기 저장을 막아 메모리 사용량을 줄이고 계산을 더 효율적으로 하기 위해서임

2.3 실습 환경 설정

생략



2.3 실습 환경 설정

- 가상 환경 생성 및 파이토치 설치(코랩 기준)
- 구글 코랩(Colab): 클라우드 기반 무료 주피터 노트북 개발 환경
- 구성/연동: 구글 드라이브 연동, 리눅스 환경, 필요 시 도커/구글 클라우드 자원 활용
- 사용 이유: 개인 PC의 성능 한계 보완
- 딥러닝은 대량 데이터 처리가 많아 고성능 PC/서버가 필요
- 개인이 고성능 환경을 갖추기 어려워 코랩 사용이 일반적
- 사용 방법: 부록 참고

2.4 파이토치 코드 맛보기



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 파이토치를 사용하여 간단한 분류 및 회귀 모델을 개발하는 방법을 살펴보자
- 예제 파일을 내려받은 후 chap02\data 폴더의 car_evaluation.csv 파일을 데이터셋으로 사용



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 데이터셋을 열어 보면 다음과 같이 특성(칼럼) 일곱 개로 구성되어 있음

1. price(자동차 가격)
2. maint(자동차 유지 비용)
3. doors(자동차 문 개수)
4. persons(수용 인원)
5. lug_capacity(수하물 용량)
6. safety(안전성)
7. output(차 상태): 이 데이터는 unacc(허용 불가능한 수준) 및 acc(허용 가능한 수준), 양호(good) 및 매우 좋은(very good, vgood) 중 하나의 값을 가짐



2.4 파이토치 코드 맛보기

- **파이토치 코드 맛보기: 설치한 라이브러리 역할 소개**
 - matplotlib: 2D/3D 그래프(플롯)를 그릴 때 주로 사용하는 시각화 패키지
 - seaborn: 데이터프레임 기반으로 통계 시각화 차트를 쉽게 만들 수 있어 데이터 분석에 많이 사용
 - scikit-learn: 분류/회귀/군집/의사결정트리 등 다양한 머신러닝 알고리즘 함수 제공 라이브러리



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 이제 필요한 라이브러리(혹은 패키지)를 호출

코드 2-1 필요한 라이브러리 호출

```
import torch
import torch.nn as nn
import numpy as np ----- 벡터 및 행렬 연산에서 매우 편리한 기능을 제공하는 파이썬 라이브러리 패키지
import pandas as pd ----- 데이터 처리를 위해 널리 사용되는 파이썬 라이브러리 패키지
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기 : 데이터 불러오기

- 예제 데이터: car_evaluation.csv
- 데이터 경로는 자신의 환경에 맞게 수정해야 함

코드 2-2 데이터 호출

```
dataset = pd.read_csv('../chap02/data/car_evaluation.csv') ----- ①
dataset.head() ----- ②
```

- ① pd.read_csv()로 ../chap02/data/car_evaluation.csv 파일을 읽어 DataFrame 생성
- ② DataFrame.head(n)로 처음 n줄을 출력해 내용 확인 (기본 n=5)
- dataset.tail(n)로 마지막 n줄을 출력해 내용 확인 가능



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 다음은 dataset.head()에 대한 출력 결과
- 참고로 인덱스는 0부터 시작

	price	maint	doors	persons	lug_capacity	safety	output
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

unacceptable

- 출력 결과에서 처음 5개 행이 단어(범주)와 숫자로 구성된 것을 확인할 수 있음
- 컴퓨터는 단어를 직접 이해하지 못하므로, 단어를 벡터로 변환하는 임베딩(embedding) 처리가 필요함



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 주어진 데이터셋을 이해하기 쉽도록 분포 형태로 시각화하여 표현하면 다음과 같음

코드 2-3 예제 데이터셋 분포

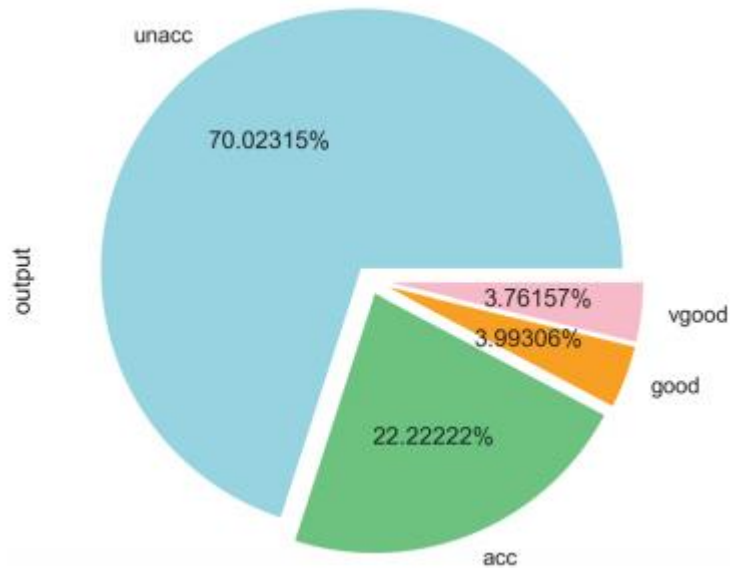
```
fig_size = plt.rcParams["figure.figsize"]
fig_size[0] = 8
fig_size[1] = 6
plt.rcParams["figure.figsize"] = fig_size
dataset.output.value_counts().plot(kind='pie', autopct='%0.05f%%',
colors=['lightblue', 'lightgreen', 'orange', 'pink'], explode=(0.05, 0.05, 0.05, 0.05))
```

- 그래프 종류를 파이차트로 지정
 - 'bar', 'line', 'hist' 등
- autopct='%0.05f%%'
 - 각 조각에 퍼센트(%) 표시
- colors=[...]
 - 각 조각의 색상을 순서대로 지정
 - explode=(0.05, 0.05, 0.05, 0.05)



2.4 파이토치 코드 맛보기

▼ 그림 2-26 예제 데이터셋 분포 결과





2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기
 - 데이터 결과 요약
 - 자동차의 대부분(약 70%)이 허용 불가능한 상태
 - 약 20%만 허용 가능한 수준 → 양호한 자동차 비율이 낮음
 - 예제 데이터 확인을 마쳤으므로 이제 전처리를 진행함



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 딥러닝 모델은 숫자 데이터(텐서)만 처리할 수 있으므로, 단어로 된 범주값은 숫자로 변환해야 함
 - 전처리의 첫 단계는 데이터가 어떤 형태(숫자/문자/범주형)인지 먼저 확인하는 것임
 - 이 예제의 컬럼들은 ‘여자/남자’처럼 범주형 데이터로 이루어져 있음
 - 따라서 다음 코드에서는 범주(단어)를 수치화된 배열 형태로 바꾸는 과정을 간단히 살펴봄
-
- 이번 장은 “맛보기”이므로 세부 구현보다 전체 흐름을 이해하는 데 집중이 필요



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 모델이 데이터를 제대로 학습하려면 먼저 전처리(preprocessing)가 필요함
- `astype('category')`를 사용하면 문자로 된 값을 범주형(category) 데이터로 변환할 수 있음
- 이후 파이토치에서 학습하려면 이 범주형 데이터를 숫자(텐서) 형태로 바꿔 모델 입력으로 사용해야 함

코드 2-4 데이터를 범주형 타입으로 변환

```
categorical_columns = ['price', 'maint', 'doors', 'persons', 'lug_capacity', 'safety']
```

예제 데이터셋 컬럼들의 목록

```
for category in categorical_columns:
```

```
    dataset[category] = dataset[category].astype('category') ----- astype() 메서드를 이용하여  
                                                                    데이터를 범주형으로 변환
```

```
price = dataset['price'].cat.codes.values ----- ①
```

```
maint = dataset['maint'].cat.codes.values
```

```
doors = dataset['doors'].cat.codes.values
```

- ① 범주형 데이터를 텐서로 변환하려면 정해진 절차가 필요함

범주형 데이터 → `dataset[category]` → 넘파이 배열(Numpy array) → 텐서(Tensor)

- 파이토치로 모델을 학습하려면 입력 데이터를 텐서 형태로 만들어야 함
- 텐서는 Numpy 배열을 이용해 생성할 수 있음



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

```
persons = dataset['persons'].cat.codes.values
lug_capacity = dataset['lug_capacity'].cat.codes.values
safety = dataset['safety'].cat.codes.values
```

```
categorical_data = np.stack([price, maint, doors, persons, lug_capacity, safety], 1) ----- ②
categorical_data[:10] ----- 합친 넘파이 배열 중 열 개의 행을 출력하여 보여 줍니다.
```

- 범주형 데이터(단어)를 숫자(NumPy 배열)로 바꾸기 위해 at.codes를 사용함
- cat.codes는 클래스↔숫자 매핑을 직관적으로 확인하기 어렵다는 단점이 있어 주의가 필요함
- ② np.stack은 두 개 이상의 NumPy 배열(객체)을 하나로 합칠 때 사용함



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

np.stack과 np.concatenate

- NumPy 배열을 합치는 대표 메서드: np.stack, np.concatenate
- 두 메서드의 핵심 차이: 차원을 유지/추가하는 방식이 다름
- np.concatenate: 선택한 축(axis)을 기준으로 배열을 이어 붙여 결합함
- np.stack: 여러 배열을 새로운 축(axis)을 만들어 쌓아 합침
- 활용 예
 - 1차원 배열들을 쌓아 2차원 배열 생성
 - 2차원 배열 여러 개를 쌓아 3차원 배열 생성
- 조건: 합칠 배열들은 차원(shape)이 동일해야 함



2.4 파이토치 코드 맛보기

- 코랩에서 다음과 같은 코드를 작성해 봄

- np.concatenate: 선택한 축(axis)을 기준으로 배열을 이어 붙여 결합함
- np.stack: 여러 배열을 새로운 축(axis)을 만들어 쌓아 합침

```
a = np.array([[1, 2], [3, 4]]) # (2행, 2열)
b = np.array([[5, 6], [7, 8]]) # (2행, 2열)
c = np.array([[5, 6], [7, 8], [9, 10]]) # (3행, 2열)
```

```
print(np.concatenate((a, b), axis=0))
```

```
print('-----')
```

```
print(np.stack((a, b), axis=0))
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 코드로 np.concatenate와 np.stack의 차이를 확인해 봄
- 임의의 NumPy 배열 a, b, c를 정의함
 - 이때 c는 다른 차원(shape)으로 정의함
- 같은 차원을 가진 a와 b에 대해
 - np.concatenate와 np.stack을 각각 적용하여 비교함

```
a = np.array([[1, 2], [3, 4]]) # (2행, 2열)
b = np.array([[5, 6], [7, 8]]) # (2행, 2열)
c = np.array([[5, 6], [7, 8], [9, 10]]) # (3행, 2열)
```

```
print(np.concatenate((a, b), axis=0))
print('-----')
print(np.stack((a, b), axis=0))
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

```
-----
[[[1 2]
  [3 4]]
 [[5 6]
  [7 8]]]
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 다음은 np.concatenate와 np.stack을 적용한 결과
- 차원이 같기 때문에 오류 없이 결과를 출력하고 있으며, np.stack의 경우에는 (2, 2, 2)로 차원이 변경된 것을 확인할 수 있음

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

```
[[[1 2]
   [3 4]]
```

```
[[5 6]
 [7 8]]]
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 이번에는 서로 다른 차원을 합쳐 보자
- 먼저 np.concatenate를 적용

```
print(np.concatenate((a, c), axis=0)) ----- shape=(5, 2)
```

- 다음과 같이 출력

```
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]]
```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 이번에는 np.stack을 적용

```
print(np.stack((a, c), axis=0))
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- np.stack은 합치려는 두 넘파이 배열의 차원이 다르기 때문에 오류가 발생

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-15-d547630d1e7e> in <module>
----> 1 print(np.stack((a, c), axis=0))

<__array_function__ internals> in stack(*args, **kwargs)

e:\Anaconda3\envs\pytorch\lib\site-packages\numpy\core\shape_base.py in
stack(arrays, axis, out)
    425     shapes = {arr.shape for arr in arrays}
    426     if len(shapes) != 1:
--> 427         raise ValueError('all input arrays must have the same shape')
    428
    429     result_ndim = arrays[0].ndim + 1

ValueError: all input arrays must have the same shape
```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 다음은 배열로 변환된 열 개의 행을 출력한 결과

```
array([[3, 3, 0, 0, 2, 1],
       [3, 3, 0, 0, 2, 2],
       [3, 3, 0, 0, 2, 0],
       [3, 3, 0, 0, 1, 1],
       [3, 3, 0, 0, 1, 2],
       [3, 3, 0, 0, 1, 0],
       [3, 3, 0, 0, 0, 1],
       [3, 3, 0, 0, 0, 2],
       [3, 3, 0, 0, 0, 0],
       [3, 3, 0, 1, 2, 1]], dtype=int8)
```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 이제 torch 모듈을 이용하여 배열을 텐서로 변환

코드 2-5 배열을 텐서로 변환

```

categorical_data = torch.tensor(categorical_data, dtype=torch.int64)
categorical_data[:10]

```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 텐서로 변환된 결과에 대한 열 개의 결과를 살펴보면 다음과 같음

```
tensor([[3, 3, 0, 0, 2, 1],
        [3, 3, 0, 0, 2, 2],
        [3, 3, 0, 0, 2, 0],
        [3, 3, 0, 0, 1, 1],
        [3, 3, 0, 0, 1, 2],
        [3, 3, 0, 0, 1, 0],
        [3, 3, 0, 0, 0, 1],
        [3, 3, 0, 0, 0, 2],
        [3, 3, 0, 0, 0, 0],
        [3, 3, 0, 1, 2, 1]])
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 마지막으로 레이블(outputs)로 사용할 칼럼에 대해서도 텐서로 변환해 줌
- 이번에는 get_dummies를 이용하여 넘파이 배열로 변환

코드 2-6 레이블로 사용할 칼럼을 텐서로 변환

```
outputs = pd.get_dummies(dataset.output) ----- ①
outputs = outputs.values
outputs = torch.tensor(outputs).flatten() ----- 1차원 텐서로 변환

print(categorical_data.shape)
print(outputs.shape)
```

- ① get_dummies는 범주형 데이터를 가변수(dummy variable)로 변환하는 함수
- 가변수 변환 의미: 문자를 0/1 형태의 숫자(원-핫 인코딩)로 바꿈
- 예시를 위해 성별(gender), 몸무게(weight), 국적(nation) 컬럼을 가진 데이터를 생성해 변환을 살펴봄



2.4 파이토치 코드 맛보기

- 다음과 같은 코드를 코랩에서 작성해 보자

```
import pandas as pd
import numpy as np

data = {
    'gender' : ['male', 'female', 'male'],
    'weight' : [72, 55, 68],
    'nation' : ['Japan', 'Korea', 'Australia']
}

df = pd.DataFrame(data)
df
```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 생성된 배열의 형태는 다음과 같음

	gender	weight	nation
0	male	72	Japan
1	female	55	Korea
2	male	68	Australia



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 성별과 국적을 숫자로 변환하기 위해 `get_dummies()`를 적용

```
pd.get_dummies(df)
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- `get_dummies()`를 적용한 결과는 다음과 같음
- 원래 숫자의 값을 가졌던 몸무게는 변화가 없고 성별과 국적만 0과 1로 변경된 것을 확인할 수 있음

	weight	gender_female	gender_male	nation_Australia	nation_Japan	nation_Korea
0	72	0	1	0	1	0
1	55	1	0	0	0	1
2	68	0	1	1	0	0

	gender	weight	nation
0	male	72	Japan
1	female	55	Korea
2	male	68	Australia



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- `ravel()`, `reshape()`, `flatten()`은 텐서의 차원을 바꿀 때 사용
- 이 메서드들은 다음과 같이 사용할 수 있음 (코랩에서 작성해 보자)

```
a = np.array([[1, 2],
              [3, 4]])
print(a.ravel())
print(a.reshape(-1))
print(a.flatten())
```

- 코드를 실행하면 다음과 같이 2차원 텐서가 1차원으로 변경되어 출력

```
[1 2 3 4]
[1 2 3 4]
[1 2 3 4]
```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 코드를 실행하면 앞에서 텐서로 변환한 범주형 데이터와 레이블에 대한 형태가 출력

```
torch.Size([1728, 6])
```

```
torch.Size([6912])
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 워드 임베딩: 의미가 비슷한 단어가 **비슷한 숫자 벡터**로 표현되도록 만드는 방법
- 임베딩 벡터의 차원이 높을수록 단어들 사이의 **세밀한 관계**를 더 잘 담을 수 있음
- 컴퓨터는 단어를 바로 처리할 수 없기 때문에, 먼저 단어/범주를 **숫자 ID**로 바꾼 뒤 그 ID를 N차원 벡터(임베딩)로 변환해 모델에 입력함
- 이때 범주형 컬럼마다 임베딩 벡터의 크기(차원)를 **미리 정해야 함**
- 임베딩 크기에는 정해진 규칙이 없지만, 보통 고유값 개수 $\div 2$ 를 많이 사용함



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 다음 코드를 이용하여 (모든 범주형 칼럼의 고유 값 수, 차원의 크기) 형태로 배열을 만들

코드 2-7 범주형 칼럼을 N차원으로 변환

```

categorical_column_sizes = [len(dataset[column].cat.categories) for column in
                             categorical_columns]
categorical_embedding_sizes = [(col_size, min(50, (col_size+1)//2)) for col_size in
                                categorical_column_sizes]

print(categorical_embedding_sizes)

```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 다음은 (모든 범주형 칼럼의 고유 값 수, 차원의 크기) 형태의 배열을 출력한 결과
[(4, 2), (4, 2), (4, 2), (3, 2), (3, 2), (3, 2)]



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 데이터셋을 훈련과 테스트 용도로 분리

코드 2-8 데이터셋 분리

```
total_records = 1728
test_records = int(total_records * .2) ----- 전체 데이터 중 20%를 테스트 용도로 사용

categorical_train_data = categorical_data[:total_records - test_records]
categorical_test_data = categorical_data[total_records - test_records:total_records]
train_outputs = outputs[:total_records - test_records]
test_outputs = outputs[total_records - test_records:total_records]
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 데이터를 훈련과 테스트 용도로 올바르게 분할했는지 확인하기 위해 레코드 개수를 출력해 보자

코드 2-9 데이터셋 분리 확인

```
print(len(categorical_train_data))
print(len(train_outputs))
print(len(categorical_test_data))
print(len(test_outputs))
```



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

- 다음은 훈련 및 테스트 용도의 레코드 개수를 출력한 결과

1383

1383

345

345



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 데이터 준비가 끝났으므로, 모델의 네트워크를 생성

코드 2-10 모델의 네트워크 생성

```
class Model(nn.Module): ----- ①
    def __init__(self, embedding_size, output_size, layers, p=0.4): ----- ②
        super().__init__() ----- ③
        self.all_embeddings = nn.ModuleList([nn.Embedding(ni, nf) for ni,
                                                nf in embedding_size])
        self.embedding_dropout = nn.Dropout(p)

        all_layers = []
        num_categorical_cols = sum((nf for ni, nf in embedding_size))
        input_size = num_categorical_cols ----- 입력층의 크기를 찾기 위해 범주형 칼럼 개수를
                                                input_size 변수에 저장
```

- ① 클래스(class) 형태로 구현되는 모델은 nn.Module을 상속받음
- ② `__init__()`은 모델에서 사용될 파라미터와 신경망을 초기화하기 위한 용도로 사용하며, 객체가 생성될 때 자동으로 호출



코드 2-10 모델의 네트워크 생성

```
class Model(nn.Module): ----- ①
    def __init__(self, embedding_size, output_size, layers, p=0.4): ----- ②
        super().__init__() ----- ③
        self.all_embeddings = nn.ModuleList([nn.Embedding(ni, nf) for ni,
                                                nf in embedding_size])

        self.embedding_dropout = nn.Dropout(p)

        all_layers = []
        num_categorical_cols = sum((nf for ni, nf in embedding_size))
        input_size = num_categorical_cols ----- 입력층의 크기를 찾기 위해 범주형 칼럼 개수를
                                                input_size 변수에 저장
```

`__init__()`에서 전달되는 매개변수는 다음과 같음

- ① self: 첫 번째 파라미터는 self를 지정해야 하며 자기 자신을 의미
예를 들어 ex라는 함수가 있을 때 self 의미는 다음 그림과 같음
- ② embedding_size: 범주형 칼럼의 임베딩 크기
- ③ output_size: 출력층의 크기
- ④ layers: 모든 계층에 대한 목록
- ⑤ p: 드롭아웃(기본값은 0.5)

③ `super().__init__()`은 부모 클래스(Model 클래스)에 접근할 때 사용하며 super는 self를 사용하지 않는 것에 주의해야 함



2.4 파이토치 코드 맛보기

```
for i in layers: ----- ④
    all_layers.append(nn.Linear(input_size, i))
    all_layers.append(nn.ReLU(inplace=True))
    all_layers.append(nn.BatchNorm1d(i))
    all_layers.append(nn.Dropout(p))
    input_size = i

all_layers.append(nn.Linear(layers[-1], output_size))
self.layers = nn.Sequential(*all_layers) ----- 신경망의 모든 계층이 순차적으로 실행되도록 모든
                                                    계층에 대한 목록(all_layers)을 nn.Sequential
                                                    클래스로 전달
```

④ 모델의 네트워크 계층을 구축하기 위해 for 문을 이용하여 각 계층을 all_layers 목록에 추가
추가된 계층은 다음과 같음

- Linear: 선형 계층(linear layer)은 입력 데이터에 선형 변환을 진행한 결과
선형 변환을 위해서는 다음 수식을 사용

$$y = Wx + b$$

(y : 선형 계층의 출력 값, W : 가중치, x : 입력 값, b : 바이어스)

선형 계층은 입력과 가중치를 곱한 후 바이어스를 더한 결과

- ReLU: 활성화 함수로 사용
- BatchNorm1d: 배치 정규화(batch normalization) 용도로 사용
- Dropout: 과적합 방지에 사용



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

```

for i in layers: ----- ④
    all_layers.append(nn.Linear(input_size, i))
    all_layers.append(nn.ReLU(inplace=True))
    all_layers.append(nn.BatchNorm1d(i))
    all_layers.append(nn.Dropout(p))
    input_size = i

all_layers.append(nn.Linear(layers[-1], output_size))
self.layers = nn.Sequential(*all_layers) ----- 신경망의 모든 계층이 순차적으로 실행되도록 모든
                                                    계층에 대한 목록(all_layers)을 nn.Sequential
                                                    클래스로 전달

def forward(self, x_categorical): ----- ⑤
    embeddings = []
    for i, e in enumerate(self.all_embeddings):
        embeddings.append(e(x_categorical[:, i]))

```

- ⑤ forward() 함수는 학습 데이터를 입력 받아서 연산을 진행
 forward() 함수는 모델 객체를 데이터와 함께 호출하면 자동으로 실행



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

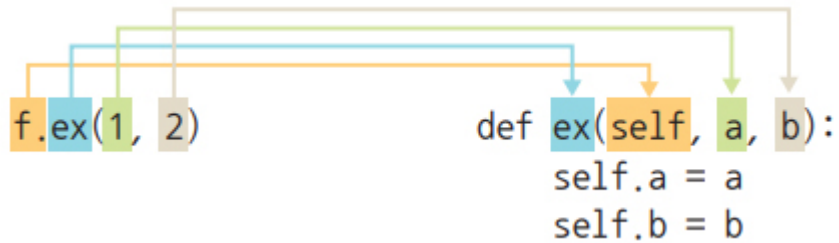
- 이 부분은 임베딩된 여러 컬럼 정보를 하나로 합쳐서, 신경망에 넣고, 최종 결과를 만드는 과정임

```
x = torch.cat(embeddings, 1) ----- 넘파이의 concatenate와 같지만 대상이 텐서가 됩니다.
x = self.embedding_dropout(x)
x = self.layers(x)
return x
```



2.4 파이토치 코드 맛보기

▼ 그림 2-31 self 의미





2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 모델 훈련을 위해 앞에서 정의했던 Model 클래스의 객체를 생성
- 객체를 생성하면서 (범주형 칼럼의 임베딩 크기, 출력 크기, 은닉층의 뉴런, 드롭아웃)을 전달
- 여기에서는 은닉층을 [200,100,50]으로 정의했지만 다른 크기로 지정하여 테스트해보는 것도 학습하는 데 도움이 될 것

코드 2-11 Model 클래스의 객체 생성

```
model = Model(categorical_embedding_sizes, 4, [200,100,50], p=0.4)
print(model)
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 코드를 실행하면 모델에 대한 구조(네트워크)를 보여 줌

```
Model(
  (all_embeddings): ModuleList(
    (0): Embedding(4, 2)
    (1): Embedding(4, 2)
    (2): Embedding(4, 2)
    (3): Embedding(3, 2)
    (4): Embedding(3, 2)
    (5): Embedding(3, 2)
  )
  (embedding_dropout): Dropout(p=0.4, inplace=False)
  (layers): Sequential(
    (0): Linear(in_features=12, out_features=200, bias=True)
    (1): ReLU(inplace=True)
    (2): BatchNorm1d(200, eps=1e-05, momentum=0.1, affine=True, track_running_
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

```
stats=True)
    (3): Dropout(p=0.4, inplace=False)
    (4): Linear(in_features=200, out_features=100, bias=True)
    (5): ReLU(inplace=True)
    (6): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (7): Dropout(p=0.4, inplace=False)
    (8): Linear(in_features=100, out_features=50, bias=True)
    (9): ReLU(inplace=True)
    (10): BatchNorm1d(50, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
    (11): Dropout(p=0.4, inplace=False)
    (12): Linear(in_features=50, out_features=4, bias=True)
)
)
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 모델을 훈련시키기 전에 손실 함수와 옵티마이저에 대해 정의해야 함
- 이번 예제는 데이터를 분류해야 하는 것으로 크로스 엔트로피(cross entropy) 손실 함수를 사용
- 또한, 옵티마이저로는 아담(Adam)을 사용

코드 2-12 모델의 파라미터 정의

```
loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 파이토치는 GPU에 최적화된 딥러닝 프레임워크
- GPU가 없다면 CPU를 사용할 수 있도록 지정해 주어야 함
- 다음은 GPU가 있다면 GPU를 사용하고, 없다면 CPU를 사용하도록 하는 코드

코드 2-13 CPU/GPU 사용 지정

```
if torch.cuda.is_available():  
    device = torch.device('cuda') ----- GPU가 있다면 GPU를 사용  
else:  
    device = torch.device('cpu') ----- GPU가 없다면 CPU를 사용
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 모델 훈련에 필요한 모든 준비가 완료
- 이제 준비된 데이터를 이용하여 모델을 학습시킴

코드 2-14 모델 학습

```
epochs = 500
aggregated_losses = []
train_outputs = train_outputs.to(device=device, dtype=torch.int64)
```

- epochs = 500 : 전체 학습을 500번 반복(에포크 수 설정)
- aggregated_losses = [] : 매 에포크의 손실값(loss)을 저장하기 위한 리스트
- train_outputs = train_outputs.to(device, dtype=torch.int64) :
 - 정답 라벨을 모델과 같은 device(CPU/GPU)로 이동
 - CrossEntropyLoss에 맞게 정수형으로 변환



코드 2-14 모델 학습

```
epochs = 500
aggregated_losses = []
train_outputs = train_outputs.to(device=device, dtype=torch.int64)

for i in range(epochs): ----- for 문은 500회 반복되며, 각 반복마다 손실 함수가 오차를 계산
    i += 1
    y_pred = model(categorical_train_data)
    single_loss = loss_function(y_pred, train_outputs)
    aggregated_losses.append(single_loss) ----- 반복할 때마다 오차를 aggregated_losses에 추가

    if i%25 == 1:
        print(f'epoch: {i:3} loss: {single_loss.item():10.8f}')
```

- `y_pred = model(categorical_train_data)` : 예측값 계산
- `single_loss = loss_function(y_pred, train_outputs)` : 예측과 정답을 비교해 손실 계산
- `aggregated_losses.append(single_loss)` : 손실을 리스트에 저장
- `if i%25 == 1: ...` : 25 에포크마다 현재 손실을 출력하여 학습 진행 상황 확인



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

```
optimizer.zero_grad()
single_loss.backward() ----- 가중치를 업데이트하기 위해 손실 함수의 backward() 메서드 호출
optimizer.step() ----- 옵티마이저 함수의 step() 메서드를 이용하여 기울기 업데이트
```

```
print(f'epoch: {i:3} loss: {single_loss.item():10.10f}') ----- 오차가 25 에포크마다 출력
```

optimizer.zero_grad() : 이전 반복에서 누적된 기울기(gradient)를 0으로 초기화
 single_loss.backward() : 손실을 기준으로 역전파(backward) 수행 → 기울기 계산
 optimizer.step() : 계산된 기울기를 사용해 모델 파라미터 업데이트



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 코드를 실행하면 25 에포크마다 출력된 오차 정보를 보여 줌

```
epoch: 1 loss: 1.63872778
epoch: 26 loss: 1.46383297
epoch: 51 loss: 1.36062038
epoch: 76 loss: 1.25486016
epoch: 101 loss: 1.11357403
epoch: 126 loss: 0.94361728
epoch: 151 loss: 0.84047800
epoch: 176 loss: 0.74985331
epoch: 201 loss: 0.70034856
epoch: 226 loss: 0.65812957
epoch: 251 loss: 0.63274646
epoch: 276 loss: 0.61346799
epoch: 301 loss: 0.60412955
epoch: 326 loss: 0.59235305
epoch: 351 loss: 0.59519970
epoch: 376 loss: 0.57206368
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

```
epoch: 401 loss: 0.57828188  
epoch: 426 loss: 0.58816069  
epoch: 451 loss: 0.57712984  
epoch: 476 loss: 0.57470286  
epoch: 500 loss: 0.5725595951
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 학습이 끝났으므로 테스트 데이터셋으로 예측을 진행해 보자
- 앞에서 준비했던 categorical_test_data 데이터셋을 모델에 적용

코드 2-15 테스트 데이터셋으로 모델 예측

```
test_outputs = test_outputs.to(device=device, dtype=torch.int64)
with torch.no_grad():
    y_val = model(categorical_test_data)
    loss = loss_function(y_val, test_outputs)
print(f'Loss: {loss:.8f}')
```

- with torch.no_grad():
 - 평가(테스트) 단계에서는 역전파가 필요 없으므로 기울기 계산/저장을 비활성화해
 - 메모리 사용량을 줄이고 속도를 높임
- y_val = model(categorical_test_data)
 - 테스트 데이터를 모델에 입력해 예측값(출력 logits)을 계산.
- loss = loss_function(y_val, test_outputs)
 - 예측값과 정답 라벨을 비교하여 테스트 손실(loss)을 계산.
- print(f'Loss: {loss:.8f}')
- 계산된 손실 값을 소수점 8자리까지 출력하여 테스트 성능을 확인.



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 코드를 실행하면 테스트 용도의 데이터셋에 대한 손실 값을 보여 줌
- 이 값은 훈련 데이터셋에서 도출된 손실 값과 비슷하므로 과적합은 발생하지 않았다고 판단할 수 있음

Loss: 0.55525565



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 이제 테스트 데이터셋을 이용했을 때 모델이 얼마나 잘 예측하는지 살펴보자
- 앞에서 모델 네트워크의 output_size에 4를 지정
- 즉, 출력층에 네 개의 뉴런이 포함되도록 지정했으므로 각 예측에는 네 개의 값이 포함될 것

코드 2-16 모델의 예측 확인

```
print(y_val[:5])
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 모델이 얼마나 잘 예측하는지 확인하기 위해 처음 다섯 개의 값을 출력하면 다음과 같음

```
tensor([[ 2.7215,  1.6601, -2.2784, -2.1693],
        [ 2.8467,  1.7512, -2.3375, -2.2104],
        [ 1.5050,  1.0164, -2.3425, -2.1092],
        [ 2.9343,  1.4001, -5.3671, -5.7565],
        [ 3.7045,  2.1831, -7.9224, -7.9769]])
```

- 값이 출력되었지만 어떤 의미인지 이해하기 어려워 보임
- 실제 출력이 0이면 인덱스 0(예 2.7215)의 값이 인덱스 1(예 1.6601)의 값보다 높아야 함



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 다음과 같은 코드를 이용하여 목록에서 가장 큰 값을 갖는 인덱스를 알아보자
- 다시 말하지만 실제 값이 아닌 인덱스를 찾는 것

코드 2-17 가장 큰 값을 갖는 인덱스 확인

```
y_val = np.argmax(y_val.cpu().numpy(), axis=1)
print(y_val[:5])
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- y_val에서 처음 다섯 개의 값이 출력

```
tensor([0, 0, 0, 0, 0])
```

- 출력 결과 모두 인덱스 0이 출력

- 즉, 인덱스가 0인 값이 인덱스가 1인 값보다 크므로 처리된 출력이 0임을 확인할 수 있음



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 마지막으로 sklearn.metrics 모듈의 classification_report, confusion_matrix, accuracy_score 클래스를 사용하여 정확도, 정밀도와 재현율을 알아보자

코드 2-18 테스트 데이터셋을 이용한 정확도 확인

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
print(confusion_matrix(test_outputs, y_val))
print(classification_report(test_outputs, y_val))
print(accuracy_score(test_outputs, y_val))
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 코드를 실행하면 모델 평가를 실행한 결과가 출력

```
[[257  2]
 [ 84  2]]
```

	precision	recall	f1-score	support
0	0.75	0.99	0.86	259
1	0.50	0.02	0.04	86
accuracy			0.75	345
macro avg	0.63	0.51	0.45	345
weighted avg	0.69	0.75	0.65	345

```
0.7507246376811594
```



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 신경망에서 필요한 모든 파라미터를 무작위로 선택했다는 것을 감안할 때 75%의 정확도는 나쁘지 않음
- 파라미터(예 훈련/테스트 데이터셋 분할, 은닉층 개수 및 크기 등)를 변경하면서 더 나은 성능을 찾아보는 것도 학습에 도움이 될 것
- 마지막으로 딥러닝 분류 모델의 성능 평가 지표를 알아보자
- 성능 평가 지표로 정확도(accuracy), 재현율(recall), 정밀도(precision), F1-스코어(F1-score)가 있음



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- 정확도를 확인하기 전에 필요한 용어들부터 살펴보자
- True Positive: 모델(분류기)이 '1'이라고 예측했는데 실제 값도 '1'인 경우
- True Negative: 모델(분류기)이 '0'이라고 예측했는데 실제 값도 '0'인 경우
- False Positive: 모델(분류기)이 '1'이라고 예측했는데 실제 값은 '0'인 경우로, Type I 오류라고도 함
- False Negative: 모델(분류기)이 '0'이라고 예측했는데 실제 값은 '1'인 경우로, Type II 오류라고도 함

		실제 정답	
		True	False
분류 결과	True	True Positive	False Positive
	False	False Negative	True Negative



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

정확도

- 전체 예측 건수에서 정답을 맞힌 건수의 비율
- 이때 맞힌 정답이 긍정(positive)이든 부정(negative)이든 상관없음

$$\frac{\text{True Positive} + \text{True Negative}}{\text{True Positive} + \text{True Negative} + \text{False Positive} + \text{False Negative}}$$

재현율

- 실제로 정답이 1이라고 할 때 모델(분류기)도 1로 예측한 비율
- 처음부터 데이터가 1일 확률이 적을 때 사용하면 좋음

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$



2.4 파이토치 코드 맛보기

- 파이토치 코드 맛보기

정밀도

- 모델(분류기)이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$



2.4 파이토치 코드 맛보기

● 파이토치 코드 맛보기

- F1-스코어는 정밀도(Precision)와 재현율(Recall)을 함께 고려한 평가 지표임
- 일반적으로 정밀도와 재현율은 서로 반비례(trade-off) 하는 경향이 있어, 한쪽이 높아지면 다른 쪽이 낮아질 수 있음
- F1-스코어는 이러한 문제를 보완하기 위해 정밀도와 재현율의 조화 평균(harmonic mean)으로 계산함
- 따라서 두 값이 균형 있게 높을 때 F1-스코어도 높게 나타남
- 이때 조화 평균은 다음 공식으로 계산함

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$